latency or API changes. The following listing presents the full test case for this function, which covers various aspects of its behavior.

Listing 8.8 Unit testing the model return values

```
import { continueConversation } from '../src/actions';
import { getSupportedModel } from '../src/utils';
import { MockLanguageModelV1 } from 'ai/test';

jest.mock('../src/utils', () => ({
  getSupportedModel: jest.fn(),
}));

describe('continueConversation', () => {
  it('should return messages and the new message
generated by the model', async () => {
    const history = [];
    const provider = 'someProvider';
    const model = 'someModel';
    const mockGeneratedText = 'Hello, world!';
    getSupportedModel.mockReturnValue(
new MockLanguageModelV1({
      doGenerate: async () => ({
        rawCall: { rawPrompt: null, rawSettings: {} },
        finishReason: 'stop',
        usage: { promptTokens: 10, completionTokens: 20 },
        text: mockGeneratedText,
      }),
    }));
    const response =
await continueConversation("Hello", history, provider, model);
    expect(getSupportedModel).toHaveBeenCalledWith(provider, model);
    expect(response.messages).toEqual(history);
    expect(response.newMessage).toEqual(mockGeneratedText);
  });
});
```

Mocks the utils module to control its behavior during testing

Mocks the return value of getSupportedModel to return an instance of MockLanguageModelV1

Asserts that getSupportedModel was called with correct arguments

While both helpers (`simulateReadableStream`, `MockLanguageModelV1`) are very useful for testing certain parts of generative AI web applications, remember that there are important caveats to consider when using mocking techniques.

Note that while the code example in the provided listing demonstrates the use of mocking techniques, another approach could be to utilize environment variables to create a mock model for testing purposes. This can help reduce the reliance on mocking techniques.

For example, you could set an environment variable `NODE_ENV` to a specific value (e.g., "testing") and then use that value to determine the appropriate model to use in your tests. This approach can provide a more realistic testing environment while still allowing for controlled testing of specific scenarios.

One significant concern is that overreliance on mocks may result in a false sense of confidence; while they can speed up testing and isolate application logic, they do not